

Name : Nishant Kumar

UID : 22BCS10430

Batch : 4

.NET 8 / C# 12 – Detailed Case Study Exercises
Value Types & Type Conversions

Exercise 1 : solution

```
using System;
class AttendanceSystem{
    static void Main(){
        int att = 68;
        int tot = 80;

        double pct = (double)att / tot * 100;

        int trunc = (int)pct;
        int round = Convert.ToInt32(Math.Round(pct));

        Console.WriteLine(pct);
        Console.WriteLine(trunc + "%");
        Console.WriteLine(round + "%");
    }
}
```

Truncation simply removes the decimal part, which can underestimate values and lead to unfair or incorrect decisions.

Rounding adjusts the value to the nearest whole number, giving a more accurate and balanced result, so it is preferred in eligibility and grading systems.

Exercise 2 : solution

```
using System;
class Program{
    static void Main(){
        int m1 = 78, m2 = 82, m3 = 91;
        double a = (m1 + m2 + m3) / 3.0;
```

```

        int r = Convert.ToInt32(Math.Round(a));
        Console.WriteLine(r);
    }
}

```

The conversion flow starts with exact types like int for input, moves to double for averaging or calculations, and finally converts to int only for display or decisions.

Precision loss happens when fractional values are removed during conversion, especially when converting from double to int. To reduce this, rounding is applied at the final step instead of truncation.

Exercise 3 : solution

```

using System;
class Program{
    static void Main(){
        decimal f = 2.5m;
        int d = 6;
        decimal t = f * d;
        double l = (double)t;
        Console.WriteLine(t);
        Console.WriteLine(l);
    }
}

```

Different data types are used based on the nature of data—int for whole numbers, decimal for money to ensure precision, and double for analytics where performance matters.

Conversions occur either implicitly when there is no risk of data loss, or explicitly using casting or conversion methods when precision or range differences exist.

Exercise 4 : solution

```

using System;
class Program{
    static void Main(){
        decimal b = 100000m;
        float r = 6.5f;
        decimal i = b * (decimal)r / 100;
        b += i;
        Console.WriteLine(b);
    }
}

```

```
}  
}
```

Safe conversions use explicit casting or conversion methods when there is a risk of data loss, such as converting float or double to decimal.

Implicit conversion may fail because C# does not allow automatic conversion between types with different precision models, as it could silently lose accuracy. Therefore, explicit casting is required to make the risk clear and intentional.

Exercise 5 : solution

```
using System;  
class Program{  
    static void Main(){  
        double c = 1299.99;  
        decimal t = 0.18m;  
        decimal p = (decimal)c + ((decimal)c * t);  
        Console.WriteLine(p);  
    }  
}
```

A conversion strategy chooses data types so calculations start with wider or approximate types and end with precise types like decimal.

Precision risks occur when converting from double or float to int or decimal, as floating-point values may lose accuracy. To reduce risk, conversions are done explicitly and rounding is applied only at the final stage.

Exercise 6 : solution

```
using System;  
class Program{  
    static void Main(){  
        short s = 302;  
        double c = s / 10.0;  
        int d = Convert.ToInt32(Math.Round(c));  
        Console.WriteLine(d);  
    }  
}
```

Overflow occurs when a value exceeds the range of its data type during conversion or calculation, leading to incorrect results or runtime errors.

Casting concerns arise when converting from a larger or floating-point type to a smaller type, as data can be lost or wrapped around. To avoid this, values should be validated and safe conversion methods or wider types should be used.

Exercise 7 : solution

```
using System;
class Program{
    static void Main(){
        double s = 86.7;
        byte g = s >= 90 ? (byte)10 : s >= 80 ? (byte)9 : (byte)8;
        Console.WriteLine(g);
    }
}
```

Validation ensures the value is within an acceptable range before conversion, preventing runtime errors or incorrect data.

Casting choices are made to avoid data loss or overflow; instead of direct casting, controlled logic or methods like Convert are used to safely transform values.

Exercise 8 : solution

```
using System;
class Program{
    static void Main(){
        long b = 5368709120;
        double g = b / (1024.0 * 1024 * 1024);
        int r = Convert.ToInt32(Math.Round(g));
        Console.WriteLine(r);
    }
}
```

Implicit conversions are automatic type conversions done by C# when there is no risk of data loss, such as converting int to double. They usually widen the data type.

Rounding methods are used when converting floating-point values to integers. Math.Round() rounds to the nearest value, Math.Floor() rounds down, and Math.Ceiling() rounds up. Rounding is preferred over truncation to maintain accuracy in calculations.

Exercise 9 : solution

```
using System;
class Program{
    static void Main(){
        int c = 5000;
        ushort m = 6000;
        bool a = c <= m;
        Console.WriteLine(a);
    }
}
```

Signed types can store negative values, but unsigned types cannot. When a signed value is converted to unsigned, a negative number can turn into a very large positive number, causing incorrect comparisons or overflows. To avoid this, I try not to mix signed and unsigned types and prefer converting both values to a wider signed type before comparison.

Exercise 10 : solution

```
using System;

class Program{
    static void Main(){
        int b = 40000;
        double a = 8500.75;
        double d = 2500.25;
        decimal n = b + (decimal)a - (decimal)d;
        Console.WriteLine(n);
    }
}
```

I design the type conversion flow by starting with the most exact type for input, such as int for counts or decimal for money. Calculations are performed using wider or appropriate types like double or decimal to avoid overflow and precision issues. Conversion to narrower types like int is done only at the final stage, using explicit casting and rounding if needed. These choices ensure data accuracy, prevent silent data loss, and make conversion risks explicit and controlled.

